AutoDSC: Automated Algorithm Selection and Tuning for Non-Stationary Data Stream Clustering

Anonymous

***Abstract*—Data stream clustering must adapt to evolving distributions without ground-truth labels, so no single algorithm or hyperparameter configuration remains effective throughout a stream. Although AutoML has transformed supervised learning, automated algorithm selection and tuning for unsupervised data streams remain largely unexplored. We introduce AutoDSC (Automated Data Stream Clustering), an AutoML framework that continuously searches the joint algorithm-and-hyperparameter space. Its *Population-Guided Stochastic Search Optimisation* (PGSSO) combines a recency-weighted elite archive with adaptive exploration, while a composite of three internal cluster-validity indices enables label-free configuration selection. Across five real-world and five synthetic streams, AutoDSC achieves the best mean rank on both Adjusted Rand Index and Cluster Mapping Measure and remains computationally competitive with stream-clustering AutoML baselines.**

***Index Terms*—data stream clustering, AutoML, algorithm selection, unsupervised learning, multi-objective optimisation**

I. Introduction

Data stream clustering is a fundamental operation across applications ranging from IoT sensor networks and network intrusion detection to social media analysis and financial monitoring [1], [2]. Unlike batch clustering, stream clustering must process data continuously while adapting to distributions that change over time. Numerous stream-clustering algorithms have been proposed [3]–[6], each effective under particular conditions. However, the appropriate algorithm and hyperparameters may themselves change as the stream evolves, because no single configuration remains optimal throughout. Existing automated solutions rely on a single evaluation metric [7], require computationally intensive offline tuning [8], or reconfigure structural components without jointly searching the algorithm-and-hyperparameter space [9]. Together, these limitations leave a gap for a fully autonomous, label-free AutoML approach to stream clustering.

In this paper, we introduce AutoDSC (Automated Data Stream Clustering), a novel AutoML framework that automatically selects and tunes a clustering algorithm online as the data distribution evolves. We make the following key contributions.

- We formalise data stream clustering as a combined algorithm selection and hyperparameter optimisation (CASH) problem [10] that must be solved repeatedly as the data distribution evolves, and present AutoDSC to solve this problem without labels or manual configuration.
- We introduce Population-Guided Stochastic Search Optimisation (PGSSO), which maintains a recency-weighted elite archive and an adaptively controlled exploration rate so that promising configurations are identified quickly and the population recovers implicitly when the distribution shifts.
- We design a normalised multi-objective quality measure combining multiple internal cluster-validity indices, providing robust label-free configuration selection.
- Across ten real-world and synthetic streams, we show that AutoDSC achieves the best mean rank on both quality metrics and the strongest throughput–memory balance among the evaluated AutoML frameworks.

The rest of the paper is organised as follows. Section II introduces the background of data stream clustering and AutoML. Section III formalises the problem. Section IV presents the AutoDSC framework. Section V describes the experimental setup. Section VI presents the empirical evaluation and analysis, and Section VIII concludes the paper.

II. Background

A. Data Stream Clustering

A data stream is an unbounded, high-velocity sequence of instances that a learner must process continuously. A stream learner should inspect each instance at most once, use bounded time per instance and bounded memory, remain ready to produce an answer, and adapt to temporal change [11]. Because the data-generating distribution evolves, streams exhibit *concept drift*: the number, shape, or location of clusters may change abruptly, gradually, or incrementally [11], [12]. A configuration that performs well at one point can therefore become suboptimal later.

Classical stream-clustering algorithms differ in how they summarise the stream. CluStream [3] maintains pyramidal time-frame statistics; density-based methods such as DenStream [4] and DBSTREAM [5] represent core, potential, and outlier micro-clusters; and k-means-based methods maintain centroids online, as in mini-batch web-scale clustering [13] and StreamKMeans [6].

Stream-clustering quality is commonly evaluated with the prequential (interleaved test-then-train) protocol [11], in which each instance tests the current model before updating it. Because ground-truth labels are unavailable during deployment, online selection relies on *internal* cluster validity indices (CVIs), computed only from observations and cluster assignments. The Silhouette Coefficient [14] measures cohesion relative to neighbouring clusters; the Davies–Bouldin Index [15] compares intra-cluster scatter with inter-cluster separation; and the Calinski–Harabasz Index [16] compares between-cluster with within-cluster dispersion. By contrast,

external indices require labels and are reserved for evaluation. The Adjusted Rand Index (ARI) [17], [18] measures chance-corrected agreement between predicted and true partitions, while the Cluster Mapping Measure (CMM) [19] additionally accounts for emerging, splitting, and moving clusters in streams.

B. Automated Machine Learning

AutoML formalises algorithm selection and hyperparameter optimisation as the Combined Algorithm Selection and Hyperparameter Optimisation (CASH) problem [10], [20], [21]. Common search methods include Bayesian optimisation, evolutionary and population-based methods, and random search [22], [23]. Clustering-specific systems combine these methods with meta-learning or Bayesian search [24]–[27], but assume full access to a dataset drawn from a fixed distribution. AutoML has also been extended to supervised streams [28]–[31]; its application to unsupervised streams, where labels cannot guide the search, remains largely unexplored.

C. Prior Work

AutoML has been applied to batch clustering [24]–[27], [32], [33] and to supervised streams [28]–[31], yet no existing system jointly addresses single-pass processing, label-free selection, and continuous distribution change. The closest work comprises three systems. ConfStream [7] maintains an ensemble of configurations and resamples around better-performing members, but selects configurations using only Silhouette Width. OnlineAutoClust (OACE) [8] applies tree-structured Parzen estimator Bayesian optimisation to clustering ensembles, but requires a computationally expensive offline tuning phase. MOStream [9], building on the modular design space studied in SESAME [34], reconfigures structural components by rule rather than searching a joint algorithm-and-hyperparameter space. AutoDSC instead combines single-pass population-guided search, a composite label-free selection criterion, and continual re-evaluation for implicit drift adaptation.

III. Problem Formulation

A data stream is an unbounded sequence $\mathcal{S} = \{x_1, x_2, \ldots\}$ in which each instance $x_t \in \mathbb{R}^d$ arrives once. AutoDSC partitions the stream into consecutive, non-overlapping windows W_s of fixed size w, where s indexes stages. Because ground-truth labels are unavailable online, clustering quality on W_s is estimated by an internal cluster-validity score $q(\cdot) \in [0, 1]$ computed from observations and assignments alone; external indices are reserved for evaluation.

Let $\mathcal{P}$ be a portfolio of stream-clustering algorithms

$$\Theta = \bigcup_{a \in \mathcal{P}} (\{a\} \times \Phi_a) \tag{1}$$

the joint space of configurations $\theta = (a, \phi)$ pairing an algorithm $a \in \mathcal{P}$ with hyperparameters $\phi \in \Phi_a$. The CASH problem for a stream is to track, at the end of each stage, the configuration

$$\theta_s^* = \arg\max_{\theta \in \Theta} \; q(\theta; W_s), \tag{2}$$

using only the data observed through stage s and bounded memory. Because drift can change θ_s^* over time, a stream CASH solver must repeatedly evaluate and select configurations.

IV. Automated Data Stream Clustering

AutoDSC addresses the online CASH problem of Section III with an AutoML framework tailored to non-stationary data stream clustering. At each stage, it processes one window W_s of w instances and then evaluates the active configurations using multiple normalised validity indices. A recency-weighted archive of high-performing configurations guides subsequent population-based search. Figure 1 illustrates this workflow.

The framework maintains a population $\mathcal{A}_s \subseteq \Theta$ of algorithms with different hyperparameter settings and serves predictions from the current best-performing configuration, called the *incumbent*. As each instance arrives, every population member first predicts its cluster and then updates incrementally with one LearnOne step. AutoDSC retains these pre-update assignments until the end of the stage for label-free scoring.

AutoDSC instantiates $q(\theta; W_s)$ as a composite of the normalised Silhouette Coefficient (NSC), Davies–Bouldin Index (NDBI), and Calinski–Harabasz Index (NCHI):

$$\begin{aligned} \text{NSC} &= (1 + \text{SC})/2, \\ \text{NDBI} &= 1 - \tanh(\text{DBI}), \\ \text{NCHI} &= \tanh(\log(1 + \text{CHI})/10)\,. \end{aligned} \tag{3}$$

All three terms are bounded to $[0, 1]$, with larger values indicating better quality. The composite score of configuration θ on W_s is

$$\mu_s(\theta) = \frac{1}{3}\big[\text{NSC}(\theta; W_s) + \text{NDBI}(\theta; W_s) + \text{NCHI}(\theta; W_s)\big]. \tag{4}$$

Equal weights avoid dataset-specific tuning and reduce dependence on any single CVI, whose reliability can vary with cluster shape and density [35], [36].

AutoDSC adjusts exploration from the change in the best population score, $\Delta\mu_s = \mu_{\text{best}}^{(s)} - \mu_{\text{prev}}$, where $\mu_{\text{best}}^{(s)} = \max_i \mu_s(\theta_i)$ and μ_{prev} is the preceding stage's best score. For stages after the first, the rate adapts as

$$\alpha_{s+1} = \begin{cases} \max(0.1,\ \alpha_s(1 - \beta_s)) & \Delta\mu_s > 0, \\ \min(1.0,\ \alpha_s(1 + \beta_s)) & \Delta\mu_s \le 0, \end{cases} \tag{5}$$

$$\beta_s = \min(\gamma,\ |r_s|), \qquad r_s = \frac{\Delta\mu_s}{|\mu_{\text{prev}}|}, \tag{6}$$

where $\gamma = 0.5$ caps the update. At the first stage, AutoDSC keeps $\alpha_0 = 0.5$ and records μ_{prev}. Thereafter, improvement reduces exploration, whereas stagnation or decline increases it and broadens the search.

The framework employs **Population-Guided Stochastic Search Optimisation (PGSSO)** (Algorithm 2) to jointly

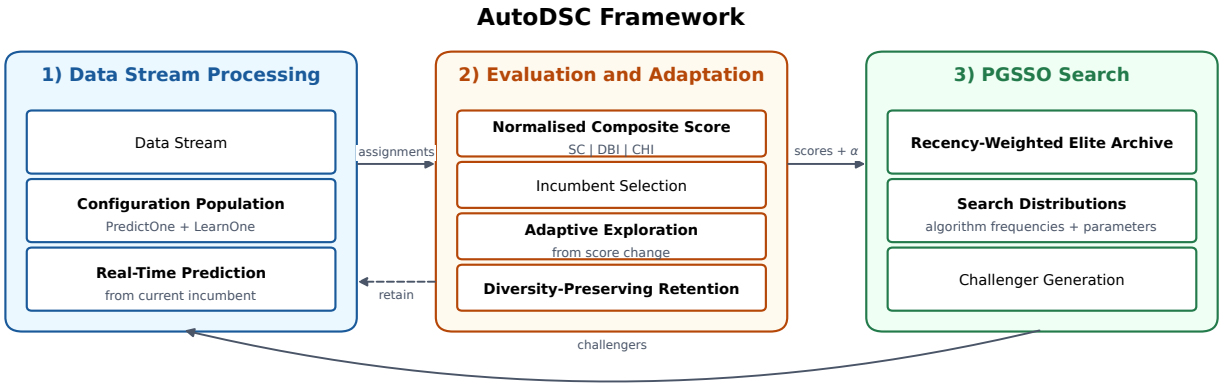


Fig. 1. Overview of AutoDSC. At each stage, the configuration population is evaluated by a composite of internal validity indices, the incumbent is re-selected, and PGSSO proposes challengers using a recency-weighted elite archive and adaptive exploration.

search over algorithms and hyperparameters. PGSSO maintains an elite archive $\mathcal{E}_s = \{(\theta_i, z_i)\}_{i=1}^{e_{\max}}$, where z_i is the score from the most recent evaluation of configuration $\theta_i = (a_i, \phi_i)$. Elites are ranked by $z_i/(1+\Delta_i)$, where Δ_i is the number of stages since the configuration was last evaluated. This recency weighting down-weights stale configurations. PGSSO then fits algorithm frequencies and, for each numerical hyperparameter j, a mean m_j and variance σ_j^2 to guide sampling.

PGSSO generates new configurations through probabilistic sampling:

$$\theta_{\text{new}} = \begin{cases} \text{Elite-Guided}(\mathcal{E}_s) & \text{with probability } 1-\alpha_s, \\ \text{Random}(\Theta) & \text{with probability } \alpha_s. \end{cases} \tag{7}$$

Elite-guided sampling first draws an algorithm with probability equal to its frequency in the elite archive, then samples that algorithm's numerical hyperparameters from a Gaussian fitted to the elites using it:

$$\phi_j \sim \mathcal{N}(m_j, \sigma_j^2) \tag{8}$$

where m_j and σ_j^2 are the mean and variance of the j-th numerical hyperparameter computed over the elite configurations that use the selected algorithm (one Gaussian per hyperparameter dimension). Samples outside a hyperparameter's defined bounds are clipped to the boundary.

At each stage boundary, AutoDSC makes room for new challengers by retaining the incumbent, the highest-scoring configurations, and then configurations selected greedily by farthest-first distance. Configurations using different algorithms have distance 1; otherwise, distance is the mean normalised difference across shared hyperparameters. The next population combines these retained configurations with newly instantiated challengers. Algorithm 1 gives the complete procedure.

A. Computational Complexity

Table I summarises AutoDSC's complexity. Here, C and M denote the per-instance time and memory of one base clusterer, p is the number of configuration parameters, and $e_{\max} = \lfloor K/2 \rfloor$. The per-stage term is dominated by the pairwise distances required by Silhouette scoring; diversity pruning adds $O(K^2 p)$ time.

V. Experimental Setup

A. Datasets

We evaluate on ten datasets comprising five real-world and five synthetic streams with varying sizes, dimensionalities, and cluster counts, as summarised in Table II. Real-world streams cover those most frequently used in prior stream clustering work, while synthetic streams probe specific drift mechanisms in isolation through controlled generators.

The five real-world streams are public benchmarks with natural class structure and distribution change.

- **Insects** [37]: optical-sensor wing-beat features of flying insects, where the species labels and feature distribution shift with ambient temperature over the recording.
- **KDD99** [38]: network-connection records from the KDD Cup 1999 intrusion-detection task, labelled as normal traffic or one of several attack types.
- **PenDigits** [38]: pen-trajectory features of handwritten digits, with one cluster per digit 0–9.
- **Shuttle** [38]: NASA Space Shuttle radiator sensor readings (Statlog), with highly imbalanced operational-state classes.
- **Vehicle** [39]: high-dimensional acoustic and seismic features from a distributed wireless sensor network, labelled by vehicle type.

The five synthetic streams isolate specific drift mechanisms under controlled conditions, holding dimensionality (d=5) and cluster count (k=5) fixed. Four are Gaussian-blob streams where the i-th cluster at time t generates points $x \sim \mathcal{N}(\mathbf{c}_i(t), \sigma_i(t)^2 I)$, differing only in how the centroid $\mathbf{c}_i(t) \in \mathbb{R}^d$ and spread $\sigma_i(t)$ evolve.

- **Expanding**: centroids are fixed ($\mathbf{c}_i(t)$=$\mathbf{c}_i^0$); within-cluster standard deviation grows linearly, $\sigma_i(t) = \sigma_0 + \lambda t$, simulating continuous cluster expansion.

Algorithm 1: AutoDSC

Input: Stream $\mathcal{S} = \{x_1, x_2, \ldots\}$, $x_t \in \mathbb{R}^d$; window size w; population size K
Output: Cluster predictions $\{y_t\}_{t=1}^{\infty}$ emitted by the incumbent θ^*

$e_{\max} \leftarrow \lfloor K/2 \rfloor$;
$\mathcal{E} \leftarrow \emptyset$, $\alpha \leftarrow 0.5$, $\mu_{\text{prev}} \leftarrow -\infty$, $s \leftarrow 0$;
$\mathcal{A}_0 \leftarrow \{\theta_i\}_{i=1}^{K}$ where $\theta_i \sim \text{Uniform}(\Theta)$;
$M_i \leftarrow \textsc{Instantiate}(\theta_i)$ for each i;
// M_i: online clusterer instantiated from $\theta_i = (a_i, \phi_i)$
$\mathcal{A} \leftarrow \mathcal{A}_0$, $W \leftarrow \emptyset$, $\theta^* \leftarrow \theta_1$;
foreach $x_t \in \mathcal{S}$ **do**
  $W \leftarrow W \cup \{x_t\}$;
  foreach $\theta_i \in \mathcal{A}$ **do**
    $\ell_{i,t} \leftarrow \textsc{PredictOne}(M_i, x_t)$;
    $M_i \leftarrow \textsc{LearnOne}(M_i, x_t)$;
  $y_t \leftarrow \ell_{i^*,t}$ where $\theta_{i^*} = \theta^*$;
  // emit prediction from current incumbent
  if $|W| = w$ **then**
    $s \leftarrow s + 1$;
    foreach $\theta_i \in \mathcal{A}$ **do**
      $L_i \leftarrow \{\ell_{i,\tau} : x_\tau \in W\}$;
      $\mu_s(\theta_i) \leftarrow \textsc{NormCVI}(W, L_i)$;
    $\mu_{\text{best}} \leftarrow \max_i \mu_s(\theta_i)$;
    $\theta^* \leftarrow \arg\max_{\theta_i \in \mathcal{A}} \mu_s(\theta_i)$;
    $\alpha \leftarrow \textsc{Adapt}(\alpha, \mu_{\text{prev}}, \mu_{\text{best}})$;
    $\mu_{\text{prev}} \leftarrow \mu_{\text{best}}$;
    $(\mathcal{E}, \mathcal{C}_{\text{new}}) \leftarrow \text{PGSSO}(\mathcal{E}, \{(\theta_i, \mu_s(\theta_i))\}, \alpha, \Theta)$;
    $\mathcal{A}_{\text{keep}} \leftarrow \textsc{SelectDiverse}(\mathcal{A}, K - |\mathcal{C}_{\text{new}}|)$;
    $\mathcal{A} \leftarrow \mathcal{A}_{\text{keep}} \cup \textsc{InstantiateAll}(\mathcal{C}_{\text{new}})$;
    $W \leftarrow \emptyset$;

Algorithm 2: PGSSO

Input: Elite archive $\mathcal{E}$; scored configurations $\{(\theta_i, z_i)\}$; exploration rate α; search space Θ; population size K; archive capacity $e_{\max}$
Output: Updated archive $\mathcal{E}$ and challengers $\mathcal{C}_{\text{new}}$

$n_{\text{gen}} \leftarrow \min\big(\max(3, \lfloor e_{\max}/2 \rfloor), K - 1\big)$;
$\mathcal{E} \leftarrow \textsc{UpdateElitePopulation}(\mathcal{E}, \{(\theta_i, z_i)\}, e_{\max})$;
// add/evict by recency-weighted score $z_i/(1 + \Delta_i)$
$\textsc{UpdateParameterDistributions}(\mathcal{E})$;
// fit $\mathcal{N}(m_j, \sigma_j^2)$ per hyperparameter j of each algorithm
$\mathcal{C}_{\text{new}} \leftarrow \emptyset$;
for $c \leftarrow 1$ **to** n_{gen} **do**
  $u \leftarrow \text{Uniform}(0, 1)$;
  if $u > \alpha$ **then**
    $\theta_{\text{new}} \leftarrow \textsc{SampleFromElite}(\mathcal{E}, \Theta)$;
  else
    $\theta_{\text{new}} \leftarrow \textsc{RandomSample}(\Theta)$;
  $\mathcal{C}_{\text{new}} \leftarrow \mathcal{C}_{\text{new}} \cup \{\theta_{\text{new}}\}$;
return $(\mathcal{E}, \mathcal{C}_{\text{new}})$;

- **Gradual**: each centroid translates at constant velocity along a fixed random direction, $\mathbf{c}_i(t) = \mathbf{c}_i^0 + v_i t$, producing smooth continuous drift throughout the stream.
- **Merge-Split**: centroids move toward the global mean $\bar{\mathbf{c}}$ during the first half of the stream and symmetrically reseparate in the second half, producing a single merge-split event.
- **Sudden**: centroid positions jump instantaneously at three evenly spaced change points and remain stationary between them, producing four stationary segments separated by abrupt concept drift.
- **Fast-RBF**: a MOA RandomRBF stream [40] with a high centroid speed parameter, generating continuously drifting clusters with complex shapes and overlapping boundaries.

B. Evaluation Protocol

AutoDSC, ConfStream, OACE, RandomSearch, and Default(Best) follow the prequential protocol. MOStream is evaluated through its released Benne interface, which in our environment requires a single fit over the standardised stream; following SESAME, its micro-clusters are then converted to macro-clusters by offline k-means refinement. We use ten independent runs per method and dataset to account for stochastic variability. Clustering quality is measured by ARI [17] and CMM [19]; ground-truth labels are used only for evaluation, never for AutoDSC training or configuration selection. We also record throughput (instances/s) and memory usage (MB). Statistical significance is assessed with the Friedman test followed by the Nemenyi post-hoc procedure [41] at $p = 0.05$.

TABLE I
PER-INSTANCE AND PER-STAGE COMPLEXITY OF AUTODSC.

Complexity	Per-Instance	Per-Stage
Time	$O(KC)$	$O(Kw^2d + K^2p)$
Space	$O\big(KM + (K + e_{\max})p + w(d + K) + w^2\big)$	

C. Baseline Methods Selection

We compare AutoDSC against five baselines spanning three categories.

- **AutoML:** ConfStream [7] and OACE [8], established frameworks that optimise algorithm selection and hyperparameters online.
- **Rule-based:** MOStream [9], which reconfigures structural components by rule without searching the joint algorithm-and-hyperparameter space.
- **Baselines:** RandomSearch [22], which shares AutoDSC's framework structure but generates configurations by purely random sampling, and Default(Best) [8], which

TABLE II
SUMMARY OF DATASETS.

Dataset Name	Type	Instances	Attributes	Clusters
Insects	Real	52,848	33	6
KDD99	Real	100,655	38	12
PenDigits	Real	10,992	16	10
Shuttle	Real	58,000	9	7
Vehicle	Real	98,528	100	3
Expanding	Synth	100,000	5	5
Gradual	Synth	100,000	5	5
Merge-Split	Synth	100,000	5	5
Sudden	Synth	100,000	5	5
Fast-RBF	Synth	100,000	5	5

TABLE III
EVALUATION PLATFORM.

Component	Description
Processor	2× AMD EPYC 7702 64-Core @ 2.0 GHz (128 cores)
L3 Cache	512 MiB
Memory	1 TiB DDR4 RAM
OS	Ubuntu 24.04.4 LTS
Kernel	Linux 6.8.0-101-generic
Runtime	Python 3.12.3, River [6]
Compiler	GCC 13.3.0

runs each base algorithm from the search space (Table IV) at its default hyperparameters and reports the best-performing one.

D. Implementation and Configuration

AutoDSC uses a population size $K = 10$ and window size $w = 1000$, with initial exploration rate $\alpha_0 = 0.5$, adapting automatically as performance feedback arrives. AutoDSC, ConfStream, OACE, and RandomSearch use the same algorithm portfolio and hyperparameter ranges (Table IV) and receive identical per-stage evaluation budgets, so their differences reflect search strategy rather than compute allotment. Baseline-specific parameters follow their original papers. All experiments were conducted on the platform detailed in Table III, with datasets and code publicly available[1].

VI. RESULTS

A. Clustering Quality

Table V compares clustering quality across ARI and CMM. AutoDSC attains the best mean score and mean rank on both metrics, with its largest advantages on streams requiring continuous adaptation. On Expanding, Gradual, and Fast-RBF, it sustains near-perfect quality while fixed default configurations degrade substantially. AutoDSC also leads on both metrics for PenDigits. The advantage is not uniform: Default(Best) achieves the highest ARI on Shuttle, MOStream leads CMM on Vehicle, and OACE leads Merge-Split. The trajectories in Figure 2 show that AutoDSC's aggregate gains on PenDigits and Expanding persist across stages rather than arising from isolated peaks.

[1]https://anonymous.4open.science/r/AutoDSC

TABLE IV
SEARCH-SPACE CONFIGURATION.

Algorithm	Hyperparameter	Type	Range/Options
CluStream	n_macro_clusters	integer	[2, 15]
	max_micro_clusters	integer	[20, 100]
	time_window	integer	[250, 1500]
	micro_cluster_r_factor	numeric	[1.0, 3.0]
DenStream	decaying_factor	numeric	[0.01, 0.2]
	beta	numeric	[0.2, 0.8]
	mu	numeric	[2.0, 8.0]
	epsilon	numeric	[0.01, 1.0]
DBSTREAM	clustering_threshold	numeric	[0.1, 2.0]
	fading_factor	numeric	[0.001, 0.05]
	intersection_factor	numeric	[0.1, 0.8]
	minimum_weight	numeric	[0.5, 5.0]
StreamKMeans	chunk_size	integer	[100, 500]
	n_clusters	integer	[2, 20]
	halflife	numeric	[0.05, 0.8]

B. Drift Behaviour

Figure 3 isolates two drift types on synthetic streams. On Merge-Split, every adaptive method incurs a temporary quality drop around the merge and subsequent split; AutoDSC remains competitive before and after these events and recovers at a rate comparable to the strongest baselines. On Expanding, AutoDSC maintains near-perfect ARI and CMM for most of the stream and degrades least as within-cluster variance grows. These results distinguish recovery from abrupt structural change from sustained adaptation under continuous drift.

C. Statistical Significance

Figure 4 shows Critical Difference (CD) diagrams from the Friedman test and Nemenyi post-hoc procedure. AutoDSC has the best mean rank on both metrics. For ARI, it significantly outperforms RandomSearch, MOStream, and Default(Best). For CMM, no pairwise difference is significant at $p = 0.05$.

D. Computational Efficiency

Figure 5 shows that AutoDSC occupies the favourable high-throughput, low-memory region of the efficiency plane, pairing the best clustering quality with a compact memory footprint. Among the AutoML methods, it is the fastest and the most memory-efficient, processing substantially more instances per second than ConfStream and OACE while consuming less memory than both. The Default(Best) baseline is naturally faster since it performs no search, and RandomSearch achieves moderate speed at the cost of quality; AutoDSC achieves a better quality-efficiency balance than either. OACE incurs the heaviest AutoML memory footprint, driven by its offline Bayesian optimisation budget, while MOStream carries the largest footprint overall owing to its structural maintenance overhead.

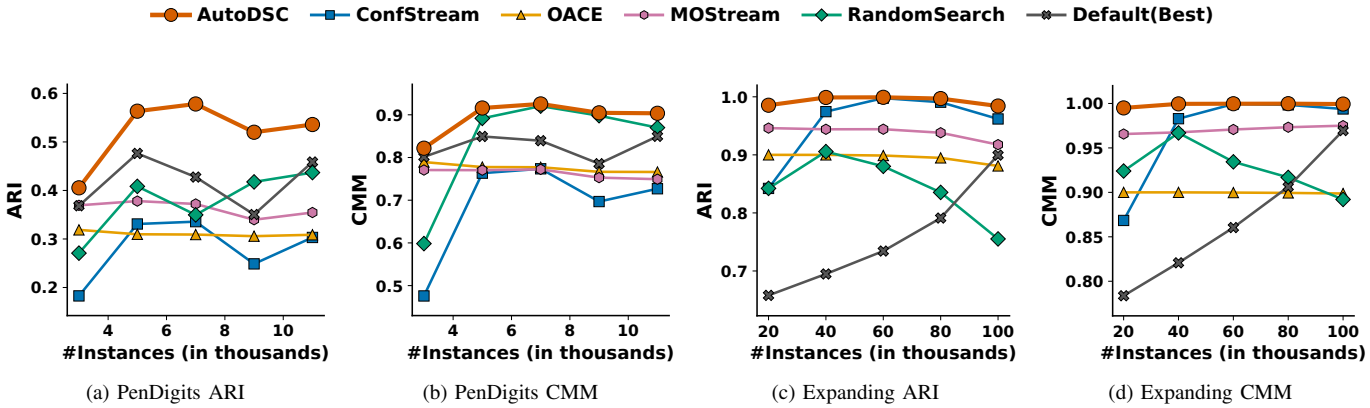
E. Ablation Study

Figure 6 reports a leave-one-out ablation across all datasets. The variants remove recency-weighted elite ranking, adaptive exploration, the composite CVI score, or diversity-preserving pruning; RandomSearch already captures the removal of PGSSO guidance. The effects are small but complementary.

TABLE V

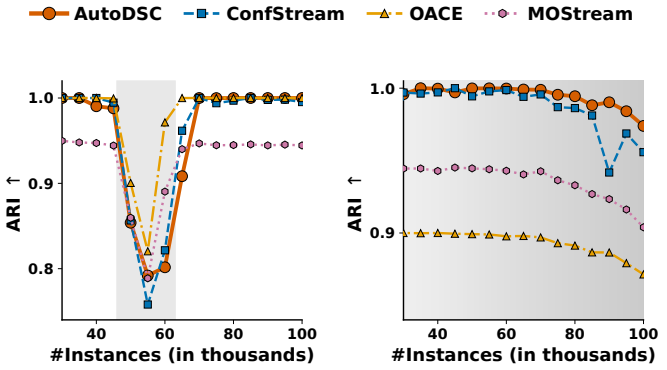
CLUSTERING QUALITY ACROSS DATASETS. ARI AND CMM (↑ HIGHER IS BETTER); BEST PER METRIC AND ROW IN **BOLD**, SECOND BEST UNDERLINED.

Dataset	AutoML						Rule-based		Baselines			
	AutoDSC		ConfStream		OACE		MOStream		RandomSearch		Default(Best)	
	ARI	CMM	ARI	CMM	ARI	CMM	ARI	CMM	ARI	CMM	ARI	CMM
Insects	**.117±.003**	.905±.024	.016±.004	.827±.019	.048±.035	**.969±.028**	.090±.045	.964±.006	.054±.007	.880±.019	.100±.002	.947±.001
KDD99	**.037±.000**	.942±.015	.037±.001	.910±.013	.036±.003	.626±.133	.018±.014	.780±.174	.033±.001	.935±.017	.037±.000	**.990±.000**
PenDigits	**.510±.012**	**.888±.014**	.271±.029	.668±.025	.311±.120	.777±.107	.363±.010	.764±.016	.367±.035	.814±.016	.412±.000	.823±.000
Shuttle	.363±.011	.900±.006	.341±.002	.887±.000	.146±.083	**.974±.034**	.002±.001	.380±.054	.270±.014	.903±.018	**.409±.000**	.886±.000
Vehicle	.320±.002	.975±.005	.303±.001	.967±.000	.312±.003	.976±.000	.335±.017	**.990±.001**	.172±.017	.977±.001	**.335±.003**	.986±.000
Expanding	**.993±.001**	**.999±.001**	.953±.028	.969±.020	.895±.298	.900±.300	.938±.111	.970±.058	.844±.017	.927±.011	.756±.103	.868±.055
Gradual	**.998±.001**	1.000±.000	.956±.042	.964±.032	.995±.014	**1.000±.000**	.978±.066	.984±.047	.921±.014	.978±.004	.759±.130	.838±.094
Merge-Split	.963±.004	.988±.001	.952±.009	.979±.007	**.985±.001**	**.999±.000**	.932±.102	.968±.062	.877±.012	.947±.009	.895±.054	.945±.034
Sudden	**.921±.001**	**.956±.001**	.904±.013	.939±.010	.855±.003	.884±.000	.774±.090	.841±.067	.878±.006	.947±.002	.646±.160	.757±.116
Fast-RBF	**.999±.001**	**1.000±.000**	.988±.001	.989±.000	.995±.008	.999±.002	.911±.109	.938±.076	.924±.009	.963±.005	.818±.096	.896±.056
Mean Score	**.622**	**.955**	.572	.910	.558	.910	.534	.858	.534	.927	.517	.894
Mean Rank	**1.40**	**2.20**	3.60	4.30	3.60	2.90	4.10	3.80	4.50	3.50	3.80	4.30



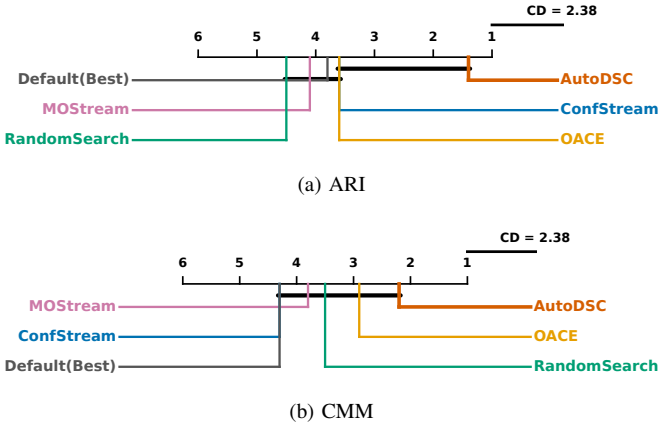
(a) PenDigits ARI (b) PenDigits CMM (c) Expanding ARI (d) Expanding CMM

Fig. 2. Prequential clustering quality across stages for a real stream (PenDigits) and a synthetic drifting stream (Expanding).



(a) Merge-Split (abrupt) (b) Expanding (continuous)

Fig. 3. Drift behaviour on synthetic streams. Shaded regions mark the drift intervals (merge and split events for Merge-Split; onset of variance growth for Expanding).



(a) ARI

(b) CMM

Fig. 4. Critical Difference diagrams for ARI and CMM using Friedman ranks and the Nemenyi post-hoc procedure ($p = 0.05$).

Removing recency weighting causes the largest ARI drop, whereas replacing the composite score has the largest effect on CMM. Diversity pruning improves both metrics, and adaptive exploration contributes modestly on average, consistent with its more targeted role during drift. The full method provides the strongest balance across ARI and CMM.

F. Population Evolution

Figure 7 shows how AutoDSC reshapes its population composition across stream stages on the Insects dataset, a real-world stream with pronounced concept drift. Rather than committing to a single algorithm, AutoDSC continuously shifts the composition between CluStream, DenStream, DBSTREAM, and StreamKMeans as the active cluster structure evolves. This emergent algorithm-switching behaviour arises purely from the recency-weighted population update, without any explicit drift

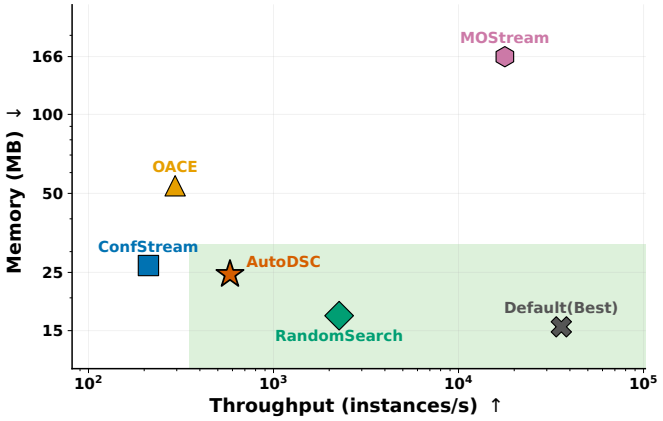


Fig. 5. Throughput-memory trade-off averaged across all datasets. The shaded green region marks the favourable high-throughput, low-memory area.

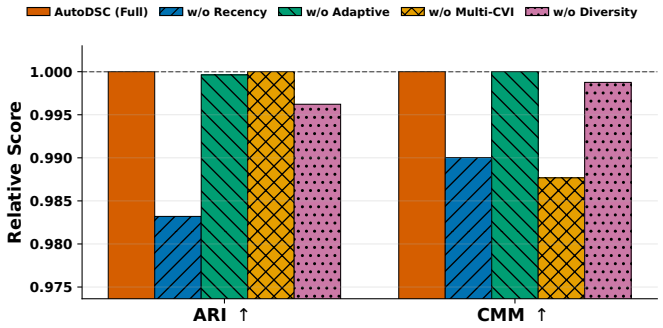


Fig. 6. Leave-one-out ablation across all datasets. ARI and CMM of each variant relative to full AutoDSC.

detector.

G. Sensitivity Analysis

Figure 8 reports AutoDSC's sensitivity to its two free hyperparameters, window size w and population size K, averaged over all datasets. For window size, quality peaks at $w = 1000$, where smaller stages give insufficient data for reliable per-stage CVI evaluation and larger stages delay adaptation to drift and reduce throughput; the quality curve is thus unimodal with a clear optimum. For population size, both ARI and CMM rise steeply from small K and saturate around $K \approx 15$, while throughput falls monotonically, so the default $K = 10$ sits at the knee of the quality-efficiency trade-off. Overall, AutoDSC is robust within a broad range of both parameters and degrades gracefully outside it, with the default configuration ($w = 1000$, $K = 10$) providing the best balance between clustering quality and computational efficiency.

VII. Discussion and Limitations

AutoDSC demonstrates that label-free, single-pass AutoML is feasible and competitive for evolving data streams, while the PGSSO search itself contributes little overhead relative to base-clusterer training. The composite scoring function reduces dependence on any one internal validity index [35],

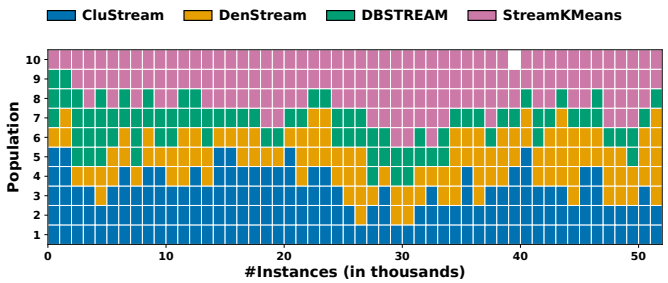
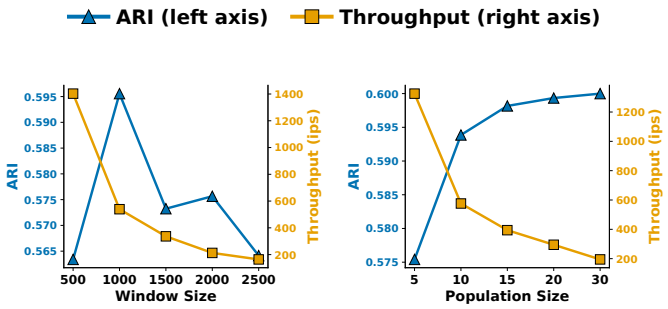


Fig. 7. Population composition over the Insects stream. Colours denote the base algorithm in each population slot (rows) and stage (columns).



(a) Window Size (b) Population Size

Fig. 8. Sensitivity to window size and population size, averaged over all datasets.

[36], and adaptive exploration broadens the search when the incumbent score stagnates or declines. AutoDSC's clearest advantages are autonomy and computational efficiency, together with significant ARI gains over untuned algorithms and random search. Its value is therefore best framed as delivering strong quality automatically rather than winning uniformly on every dataset and metric.

Several limitations bound the scope of our claims. First, with ten datasets, AutoDSC's advantage over the strongest AutoML competitors, ConfStream and OACE, is not statistically significant, although AutoDSC has the best mean rank and significantly outperforms untuned algorithms and random search on ARI. Second, the evaluation targets numerical tabular streams; sparse text, multimodal, and image streams would require domain-specific base clusterers. Third, AutoDSC exposes two framework-level hyperparameters, window size and population size, although the sensitivity analysis shows stability across broad ranges. Fourth, while PGSSO supports categorical parameters, the current portfolio contains only numerical and integer choices. Finally, internal validity indices estimate rather than observe clustering quality online, so a misleading score can temporarily affect configuration selection until later stages provide new evidence.

VIII. Conclusion and Future Work

We presented AutoDSC, a single-pass AutoML framework that uses PGSSO to propose stream-clustering configurations and a composite internal criterion to select among them

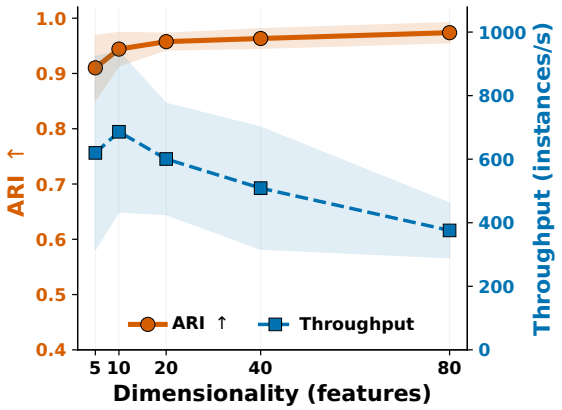


Fig. 9. Dimensionality scalability of AutoDSC. ARI remains stable while throughput decreases as feature count grows.

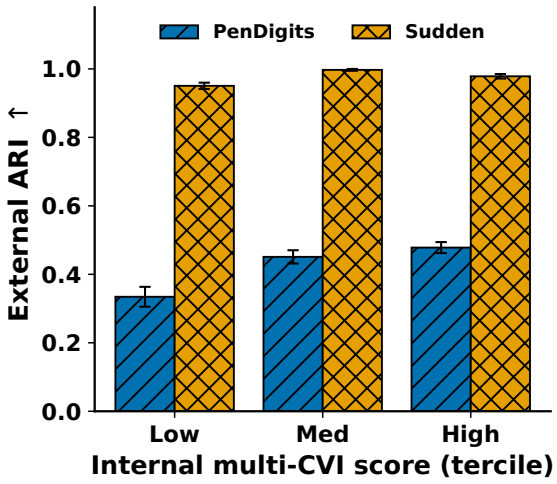


Fig. 10. Label-free selection validity on PenDigits and Sudden. External ARI rises with the incumbent's internal multi-CVI score on PenDigits and remains high on Sudden.

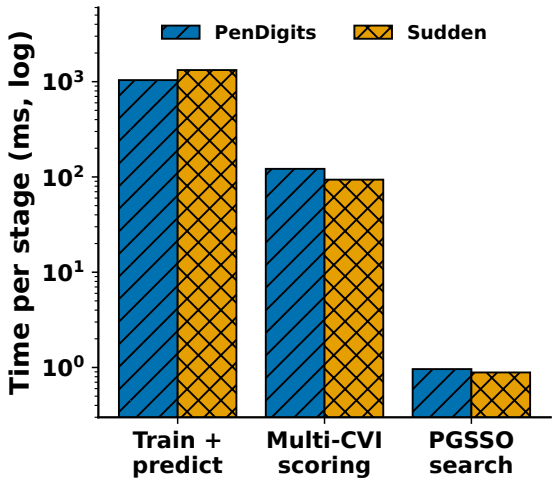


Fig. 11. Per-stage search overhead decomposition on log scale. PGSSO search is negligible relative to base-clusterer training.

without labels. Across ten streams, AutoDSC achieves the best mean rank on ARI and CMM, significantly outperforms untuned algorithms and random search on ARI, and remains computationally competitive with AutoML stream clusterers. These results show that continuous, label-free algorithm selection and hyperparameter optimisation are feasible for evolving streams. Future work will investigate partial label feedback, richer selection objectives, and portfolios with categorical structural choices.

Appendix A
Dimensionality Scalability

Figure 9 shows AutoDSC's behaviour as feature dimensionality grows on a controlled synthetic stream. Clustering quality remains stable across the tested range, while throughput decreases gradually. Thus, within this setting, increasing dimensionality affects computational cost more strongly than ARI or CMM.

Appendix B
Label-Free Selection Validity

Figure 10 validates that AutoDSC's label-free internal quality measure tracks external quality. Bucketing stages by the incumbent's multi-CVI score tercile shows that external ARI rises consistently with the internal score on PenDigits, and that on Sudden the incumbent already attains near-perfect quality across all terciles. Additionally, although AutoDSC never observes the true number of clusters, the incumbent's mean per-stage cluster count closely matches the ground truth on both streams, indicating that the label-free selection mechanism implicitly recovers the correct cluster structure.

Appendix C
Search Overhead Decomposition

Figure 11 decomposes AutoDSC's per-stage wall-clock into base-clusterer training and prediction, multi-CVI scoring, and the PGSSO search. The three components span several orders of magnitude on a log scale, with base-clusterer training dominating and the AutoML search machinery contributing a negligible fraction of the total time. This confirms that the cost of automation is dwarfed by the clustering it automates, explaining AutoDSC's competitive throughput despite continuously searching the configuration space.

Appendix D
Adaptive Exploration Rate Trajectory

Figure 12 shows AutoDSC's per-stage exploration rate α_s alongside the incumbent CVI score on Merge-Split. During stable phases, α_s remains low; it rises as the score drops around each drift event and decreases after the incumbent adapts. The repeated response at the merge and split shows that the mechanism reacts to performance change without an explicit drift detector.

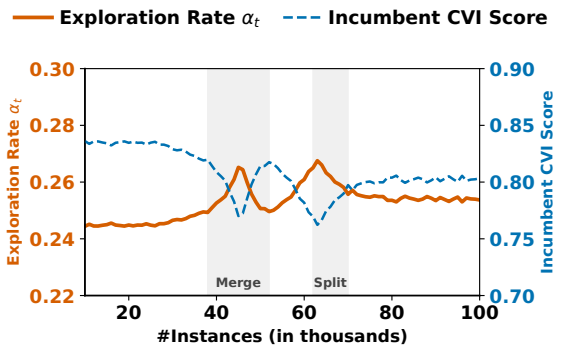


Fig. 12. Per-stage exploration rate α_s and incumbent CVI score on Merge-Split. Shaded bands mark the merge and split intervals.

APPENDIX E
SUPPLEMENTARY QUALITY METRICS ACROSS DATASETS

Table VI reports NMI [42] and Purity [43] averaged across datasets. AutoDSC has the highest mean score and best mean rank on both metrics, with OACE second. This agreement with the ARI and CMM results shows that AutoDSC's aggregate advantage is not specific to either primary metric.

TABLE VI
SUPPLEMENTARY QUALITY METRICS ACROSS DATASETS. MEAN NMI AND PURITY (↑ HIGHER IS BETTER); BEST IN **BOLD**, SECOND BEST UNDERLINED.

Method	NMI	NMI Rank	Purity	Purity Rank
AutoDSC	**.655**	**1.50**	**.853**	**2.10**
ConfStream	.610	3.30	.804	4.00
OACE	.629	2.80	.848	2.80
MOStream	.585	4.00	.803	4.10
RandomSearch	.609	4.40	.843	3.60
Default(Best)	.578	5.00	.773	4.40

REFERENCES

[1] L. Yang and A. Shami, "IoT data analytics in dynamic environments: From an automated machine learning perspective," *Engineering Applications of Artificial Intelligence*, vol. 116, p. 105366, 2022.

[2] M. Bahri, A. Bifet, J. Gama, H. M. Gomes, and S. Maniu, "Data stream analysis: Foundations, major tasks and tools," *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 11, no. 3, p. e1405, 2021.

[3] C. C. Aggarwal, J. Han, J. Wang, and P. S. Yu, "A framework for clustering evolving data streams," in *Proceedings of the 29th International Conference on Very Large Data Bases (VLDB)*. Morgan Kaufmann, 2003, pp. 81–92.

[4] F. Cao, M. Ester, W. Qian, and A. Zhou, "Density-based clustering over an evolving data stream with noise," in *Proceedings of the Sixth SIAM International Conference on Data Mining (SDM)*. SIAM, 2006, pp. 328–339. [Online]. Available: https://epubs.siam.org/doi/abs/10.1137/1.9781611972764.29

[5] M. Hahsler and M. Bolaños, "Clustering data streams based on shared density between micro-clusters," *IEEE Transactions on Knowledge and Data Engineering*, vol. 28, no. 6, pp. 1449–1461, 2016.

[6] J. Montiel, M. Halford, S. M. Mastelini, G. Bolmier, R. Sourty, R. Vaysse, A. Zouitine, H. M. Gomes, J. Read, T. Abdessalem *et al.*, "River: Machine learning for streaming data in Python," *Journal of Machine Learning Research*, vol. 22, no. 110, pp. 1–8, 2021. [Online]. Available: http://jmlr.org/papers/v22/20-1381.html

[7] M. Carnein, H. Trautmann, A. Bifet, and B. Pfahringer, "confstream: Automated algorithm selection and configuration of stream clustering algorithms," in *International Conference on Learning and Intelligent Optimization (LION)*. Springer, 2020, pp. 80–95.

[8] R. El Shawi and D. Rozgonjuk, "OnlineAutoClust: A framework for online automated clustering," in *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management (CIKM)*. ACM, 2023, pp. 3870–3874.

[9] Z. Wang, X. Wang, and S. Zhang, "MOStream: A modular and self-optimizing data stream clustering algorithm," in *2024 IEEE International Conference on Data Mining (ICDM)*. IEEE, 2024, pp. 500–509.

[10] F. Hutter, L. Kotthoff, and J. Vanschoren, *Automated Machine Learning: Methods, Systems, Challenges*. Springer Nature, 2019.

[11] A. Bifet, R. Gavaldà, G. Holmes, and B. Pfahringer, *Machine Learning for Data Streams with Practical Examples in MOA*. MIT Press, 2018. [Online]. Available: https://moa.cms.waikato.ac.nz/book/

[12] C. C. Aggarwal, *Data Streams: Models and Algorithms*, ser. Advances in Database Systems. Springer, 2007, vol. 31.

[13] D. Sculley, "Web-scale K-means clustering," in *Proceedings of the 19th International Conference on World Wide Web (WWW)*. ACM, 2010, pp. 1177–1178.

[14] P. J. Rousseeuw, "Silhouettes: A graphical aid to the interpretation and validation of cluster analysis," *Journal of Computational and Applied Mathematics*, vol. 20, no. 1, pp. 53–65, 1987.

[15] D. L. Davies and D. W. Bouldin, "A cluster separation measure," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-1, no. 2, pp. 224–227, 1979.

[16] T. Caliński and J. Harabasz, "A dendrite method for cluster analysis," *Communications in Statistics – Theory and Methods*, vol. 3, no. 1, pp. 1–27, 1974.

[17] L. Hubert and P. Arabie, "Comparing partitions," *Journal of Classification*, vol. 2, no. 1, pp. 193–218, 1985.

[18] W. M. Rand, "Objective criteria for the evaluation of clustering methods," *Journal of the American Statistical Association*, vol. 66, no. 336, pp. 846–850, 1971.

[19] H. Kremer, P. Kranen, T. Jansen, T. Seidl, A. Bifet, G. Holmes, and B. Pfahringer, "An effective evaluation measure for clustering on evolving data streams," in *Proceedings of the 17th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, ser. KDD '11. New York, NY, USA: Association for Computing Machinery, 2011, pp. 868–876.

[20] C. Thornton, F. Hutter, H. H. Hoos, and K. Leyton-Brown, "Auto-WEKA: Combined selection and hyperparameter optimization of classification algorithms," in *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. ACM, 2013, pp. 847–855.

[21] M. Feurer, A. Klein, K. Eggensperger, J. Springenberg, M. Blum, and F. Hutter, "Efficient and robust automated machine learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28. Curran Associates, 2015, pp. 2962–2970.

[22] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *Journal of Machine Learning Research*, vol. 13, no. 1, pp. 281–305, 2012.

[23] N. Hansen and A. Ostermeier, "Completely derandomized self-adaptation in evolution strategies," *Evolutionary Computation*, vol. 9, no. 2, pp. 159–195, 2001.

[24] D. Tschechlov, M. Fritz, and H. Schwarz, "AutoML4Clust: Efficient AutoML for clustering analyses," in *Proceedings of the 24th International Conference on Extending Database Technology (EDBT)*. OpenProceedings.org, 2021, pp. 343–348.

[25] D. Treder-Tschechlov, M. Fritz, H. Schwarz, and B. Mitschang, "ML2DAC: Meta-learning to democratize AutoML for clustering analysis," *Proceedings of the ACM on Management of Data*, vol. 1, no. 2, Jun. 2023.

[26] R. ElShawi, H. Lekunze, and S. Sakr, "cSmartML: A meta learning-based framework for automated selection and hyperparameter tuning for clustering," in *2021 IEEE International Conference on Big Data (Big Data)*. IEEE, 2021, pp. 1119–1126.

[27] R. ElShawi and S. Sakr, "TPE-AutoClust: A tree-based pipeline ensemble framework for automated clustering," in *2022 IEEE International Conference on Data Mining Workshops (ICDMW)*. IEEE, 2022, pp. 1144–1153.

[28] Q. Wu, C. Wang, J. Langford, P. Mineiro, and M. Rossi, "ChaCha for online AutoML," in *Proceedings of the 38th International*

Conference on Machine Learning (ICML), ser. Proceedings of Machine Learning Research, M. Meila and T. Zhang, Eds., vol. 139. PMLR, 18–24 Jul 2021, pp. 11 263–11 273. [Online]. Available: https://proceedings.mlr.press/v139/wu21d.html

[29] B. Celik, P. Singh, and J. Vanschoren, "Online AutoML: An adaptive AutoML framework for online learning," *Machine Learning*, vol. 112, no. 6, pp. 1897–1921, 2023.

[30] N. Verma, A. Bifet, B. Pfahringer, and M. Bahri, "ASML: A scalable and efficient AutoML solution for data streams," in *Proceedings of the Third International Conference on Automated Machine Learning (AutoML)*, ser. Proceedings of Machine Learning Research, K. Eggensperger, R. Garnett, J. Vanschoren, M. Lindauer, and J. R. Gardner, Eds., vol. 256. PMLR, 09–12 Sep 2024, pp. 11/1–26. [Online]. Available: https://proceedings.mlr.press/v256/verma24a.html

[31] ——, "Auto-Reg: A dynamic AutoML framework for streaming regression," in *Pacific-Asia Conference on Knowledge Discovery and Data Mining (PAKDD)*. Springer, 2025, pp. 245–256.

[32] Y. Liu, S. Li, and W. Tian, "AutoCluster: Meta-learning based ensemble method for automated unsupervised clustering," in *Advances in Knowledge Discovery and Data Mining (PAKDD 2021)*. Springer, 2021, pp. 246–258.

[33] M. C. da Silva, G. M. Tavares, E. Medvet, and S. B. Junior, "Problem-oriented AutoML in clustering," *arXiv preprint*, vol. arXiv:2409.16218, 2024, arXiv:2409.16218. [Online]. Available: https://arxiv.org/abs/2409.16218

[34] X. Wang, Z. Wang, Z. Wu, S. Zhang, X. Shi, and L. Lu, "Data stream clustering: An in-depth empirical study," *Proceedings of the ACM on Management of Data*, vol. 1, no. 2, 2023.

[35] O. Arbelaitz, I. Gurrutxaga, J. Muguerza, J. M. Pérez, and I. Perona, "An extensive comparative study of cluster validity indices," *Pattern Recognition*, vol. 46, no. 1, pp. 243–256, 2013.

[36] Y. Liu, Z. Li, H. Xiong, X. Gao, and J. Wu, "Understanding of internal clustering validation measures," in *2010 IEEE International Conference on Data Mining (ICDM)*. IEEE, 2010, pp. 911–916.

[37] V. M. Souza, D. M. dos Reis, A. G. Maletzke, and G. E. Batista, "Challenges in benchmarking stream learning algorithms with real-world data," *Data Mining and Knowledge Discovery*, vol. 34, no. 6, pp. 1805–1858, 2020.

[38] D. Dua and C. Graff, "UCI machine learning repository," University of California, Irvine, School of Information and Computer Sciences, 2019. [Online]. Available: http://archive.ics.uci.edu/ml

[39] M. Duarte and Y. H. Hu, "Vehicle classification in distributed sensor networks," *Journal of Parallel and Distributed Computing*, vol. 64, no. 7, pp. 826–838, 2004.

[40] A. Bifet, G. Holmes, R. Kirkby, and B. Pfahringer, "MOA: Massive online analysis," *Journal of Machine Learning Research*, vol. 11, pp. 1601–1604, 2010.

[41] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006.

[42] A. Strehl and J. Ghosh, "Cluster ensembles: A knowledge reuse framework for combining multiple partitions," *Journal of Machine Learning Research*, vol. 3, pp. 583–617, 2002.

[43] M. Steinbach, G. Karypis, and V. Kumar, "A comparison of document clustering techniques," in *KDD Workshop on Text Mining*, 2000, pp. 525–526.